

Build Your First OpenConstructionERP Module

Intern track — learn the “no-fork” module flow on a stock OCE. You will stand up OpenConstructionERP, add your own module with its own database table, router and page, and see it appear at `/api/v1/<you>`.

Ground rules (read first)

Do NOT clone or edit the team repo (achi-erp). That is production. You practise on a throwaway copy of the **official** OpenConstructionERP, installed from PyPI. When you understand the flow, you will build features into the real module with a mentor.

- You never fork or rebuild OCE's frontend. You add a backend module + your own HTML page.
- OCE is installed **stock** from PyPI. Your module is copied into the installed package — upstream files are untouched.
- Everything runs locally with an **embedded PostgreSQL** — no Docker, no database to set up.

1. Set up a stock OCE (WSL / Linux / macOS)

Use **uv** so you get Python 3.12 (OCE's target) without touching your system Python:

```
# 1. uv (manages Python 3.12; no sudo)
curl -LsSf https://astral.sh/uv/install.sh | sh
export PATH="$HOME/.local/bin:$PATH"

# 2. a practice folder + venv on Python 3.12
mkdir ~/oce-practice && cd ~/oce-practice
uv venv --python 3.12

# 3. install stock OpenConstructionERP from PyPI
uv pip install "openconstructionerp==11.9.0"

# 4. run it (embedded PostgreSQL boots itself)
uv run openconstructionerp serve --host 127.0.0.1 --port 8080 --data-dir ./data
```

Open **`http://localhost:8080`** and sign in with the demo account printed in the terminal (default `demo@openconstructionerp.com / DemoPass1234!`).

2. The module contract (how the loader finds you)

OCE scans `app/modules/*/manifest.py` on startup and auto-mounts each module's router. No upstream file is edited. The rules that actually matter:

- **Folder name** = `manifest.name` minus the `oe_` prefix. So `oe_hello` → `modules/hello/`.
- **manifest.py** must export a **lowercase** manifest (UPPERCASE is ignored). `category` must **not** be `"core"`.
- **models.py** is imported before `create_all`. Import from `app.database import Base`. **Namespace every table** (e.g. `hello_*`).
- **router.py** must export `router`. It auto-mounts at `/api/v1/<kebab-name>/`.
- `Base` already gives you `id`, `created_at`, `updated_at` — don't redeclare them.

3. Write your module

Create `modules/hello/` in your practice folder with four files.

manifest.py

```
from app.core.module_loader import ModuleManifest
manifest = ModuleManifest()
```

```

    name="oe_hello", version="0.1.0", display_name="Hello",
    description="My first module.", author="Intern",
    category="community", # never 'core'
    depends=[], auto_install=False, enabled=True,
)

```

models.py — one namespaced table

```

import uuid
from sqlalchemy import String, Text
from sqlalchemy.orm import Mapped, mapped_column
from app.database import Base

class HelloNote(Base):
    __tablename__ = "hello_note" # namespaced!
    id: Mapped[str] = mapped_column(String(36), primary_key=True,
                                     default=lambda: str(uuid.uuid4()))
    text: Mapped[str] = mapped_column(Text, nullable=False, default="")

```

schemas.py

```

from pydantic import BaseModel
class NoteIn(BaseModel): text: str
class NoteOut(BaseModel):
    model_config = {"from_attributes": True}
    id: str; text: str

```

router.py — endpoints + your own page

```

from fastapi import APIRouter
from fastapi.responses import HTMLResponse
from sqlalchemy import select
from app.dependencies import SessionDep, CurrentUserId
from .models import HelloNote
from .schemas import NoteIn, NoteOut
router = APIRouter()

@router.get("/info")
def info(): return {"module": "oe_hello", "ok": True}

@router.get("/notes/", response_model=list[NoteOut])
async def list_notes(session: SessionDep, _u: CurrentUserId):
    rows = (await session.execute(select(HelloNote))).scalars().all()
    return [NoteOut.model_validate(r) for r in rows]

@router.post("/notes/", response_model=NoteOut, status_code=201)
async def add_note(data: NoteIn, session: SessionDep, _u: CurrentUserId):
    n = HelloNote(text=data.text); session.add(n)
    await session.commit(); await session.refresh(n)
    return NoteOut.model_validate(n)

@router.get("/ui", response_class=HTMLResponse, include_in_schema=False)
def ui():
    # same origin as the SPA -> reuse the JWT it already stored
    return HTMLResponse('''<!doctype html><meta charset=utf-8>
<h2>Hello notes</h2><script>
const T=localStorage.getItem("oe_access_token");
fetch("/api/v1/hello/notes/",{headers:{Authorization:"Bearer "+T}})
.then(r=>r.json()).then(d=>document.body.append(JSON.stringify(d)));
</script>''')

```

4. Load it and verify

The loader only scans the installed package, so copy your module in, then restart:

```

# stop the server (Ctrl+C), then:

```

```
APP=$(uv run python -c "import app,pathlib;print(pathlib.Path(app.__file__).parent)")
cp -r modules/hello "$APP/modules/hello"
uv run openconstructionerp serve --host 127.0.0.1 --port 8080 --data-dir ./data

# in the boot log you should see:
#   Mounted router for oe_hello at /api/v1/hello

# verify:
curl -s localhost:8080/api/v1/hello/info      # {"module":"oe_hello","ok":true}
# your page: http://localhost:8080/api/v1/hello/ui (log in at :8080 first)
```

The `hello_note` table is created automatically by `create_all` on that first boot. That is the whole loop: manifest → models → router → copy in → restart → live at `/api/v1`.

5. Rules to keep you safe

- **Namespace tables** (`hello_*`). A clash with an upstream `oe_*` table is painful — schema is create-all, no migrations.
- Depend on upstream modules via **optional_depends**, and **never import upstream at module top-level** (do it inside functions) — so a disabled upstream module can't break yours.
- Serve your **own** HTML page from your router; read the token from `localStorage[ 'oe_access_token' ]`. Same origin = no second login.
- If a page says “not signed in”: log into the app on the **same host** you view it on (localhost and 127.0.0.1 are different origins).

When you can build, load and hit your own module end-to-end, you understand the pattern the real ACHI module uses — and you are ready to work in it **with a mentor**. Until then, keep your experiments in `~/oce-practice`.